

UFUG 2602

Recursion

Tengfei Chang, Jia Li, Xihu Zheng, Zijun Gong

IoT, DSA, INTR Thrusts
Hong Kong University of Science and Technology (Guangzhou)

Outline

- Recursion
 - **Introduction**
 - Real-World Example
 - Definition
 - Examples
 - Binary Search
 - Fibonacci Numbers

Real-World Example: Russian nesting dolls



- Russian nesting dolls are a set of toys where each doll contains a smaller doll inside, continuing until the smallest doll.
- Opening the first nesting doll:
 1. If there is another doll inside, continue opening.
 2. If there is no doll inside, stop.



https://www.youtube.com/watch?v=93_iFq6rBy8

Introduction

- Definition of **recursion** from Wikipedia:
 - Recursion is a method of solving a computational problem where the solution depends on solutions to smaller instances of the same problem.
 - Recursion solves such recursive problems by using functions that call themselves from within their own code.
- When dealing with a subproblem, you can consider it solved and use the subproblem's output directly to continue the algorithm design.
- Consider the following equation
 - $f(n) = n * f(n-1)$, if $n \geq 1$
 - $f(n) = 1$, if $n = 0$
- What is the value of $f(3)$?

Implementing $f()$ as a Recursive Function

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

In algorithm design, you can just consider $f(n-1)$ is solved.
But What happens when a function calls "itself"?

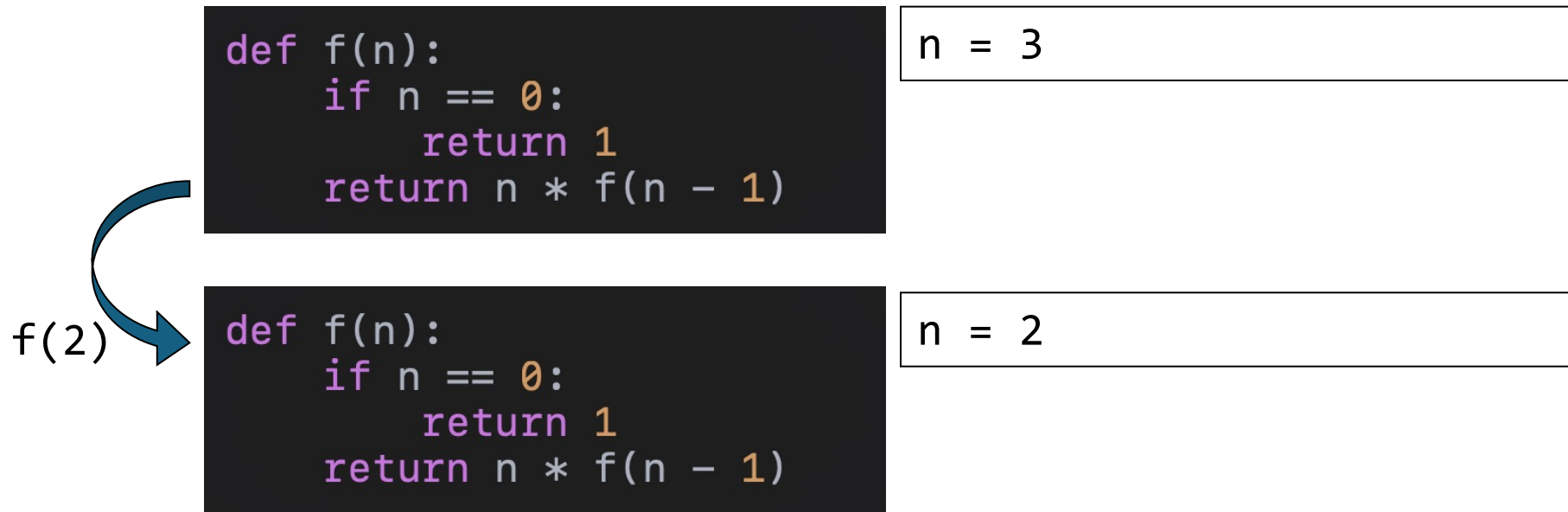
f(3)

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 3

Initially, main() calls f(3).

In this function call, n is set to 3 from main().



Conceptually, the function is not actually calling "itself" but a "clone" of itself.

Each "clone" of the function has its own local variables.

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 3

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 2

f(1)

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 1

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 3

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 2

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 1

f(0)



```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 0

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 3

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 2

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 1

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 0

returns 1



```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

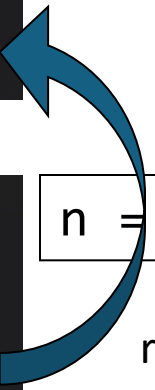
n = 3

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 2

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 1



returns
 $1 * f(0)$
 $= 1 * 1$
 $= 1$

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 3

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

n = 2

returns
 $2 * f(1)$
 $= 2 * 1$
 $= 2$

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

`n = 3`

returns
 $3 * f(2)$
 $= 3 * 2$
 $= 6$

Finally, 6 is returned.

Introduction

- There are two main components in a recursive function:
- **Base case (terminating case):**
 - The function produces a result trivially (without recurring) and returns.
- When a recursion cannot terminate, the computer will eventually run out of "stack space" in the memory and generate a runtime error called "stack overflow".

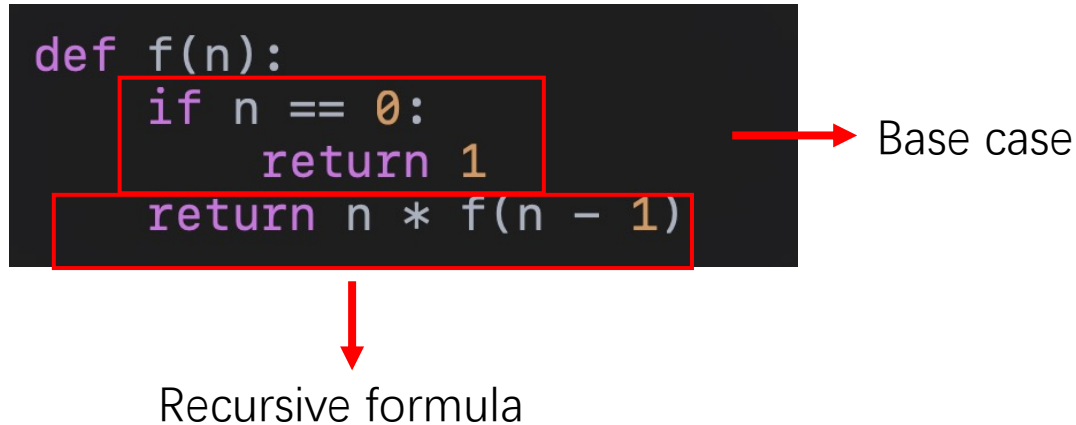
```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```

→ Base case

Introduction

- There are two main components in a recursive function:
- **Base case (terminating case):**
 - The function produces a result trivially (without recurring) and returns.
- **Recursive formula:**
 - Breaking down current problem into smaller subproblems.

```
def f(n):  
    if n == 0:  
        return 1  
    return n * f(n - 1)
```



The diagram illustrates the components of a recursive function. A red box highlights the base case `if n == 0: return 1`, with a red arrow pointing to the text "Base case". Another red box highlights the recursive formula `return n * f(n - 1)`, with a red arrow pointing down to the text "Recursive formula".

Exercise 1 — Power Function x^n

- Define a recursive function to compute: x^n
- What is the base case?
- What is the recursive formula?

Exercise 2 – Count Digits

- Write recursion for countDigits(n) (positive integer).
- What is the base case?
- What is the recursion formula?
- and compute countDigits(58391).

Exercise 3 – Reverse String

- Write recursion for reverseString(s).
- What is the base case?
- What is the recursion formula?
- Compute reverseString("DATA")

Exercise 4 – Min in Array

- Let `minArray(arr,n)` return min of first `n` elements.
- What is the base case?
- What is the recursion formula?

Outline

- Recursion
 - Introduction
 - Real-World Example
 - Definition
 - **Examples**
 - Binary Search
 - Fibonacci Numbers

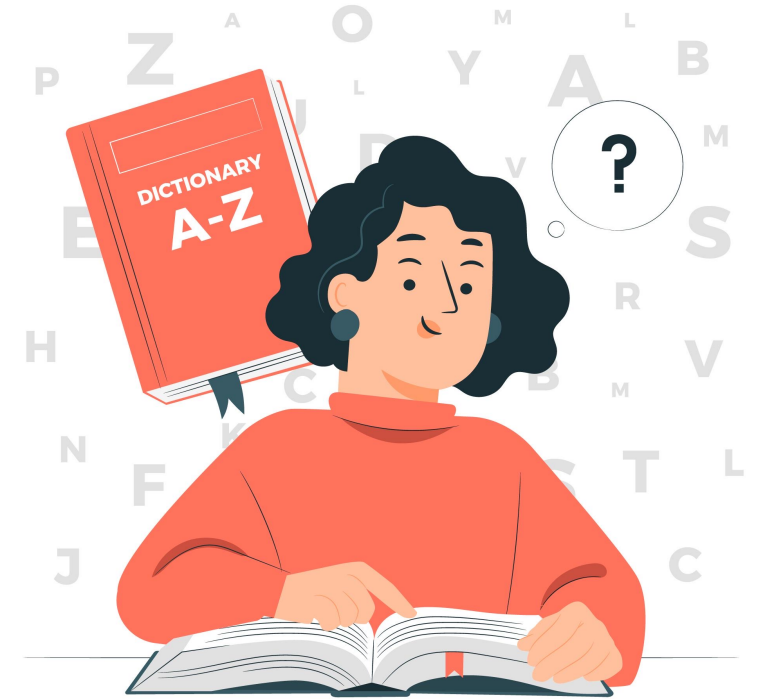
Binary Search– An example



Looking Up a Word in a Dictionary

When you **search for a word in a dictionary**, you don't start from the first page and go through every word sequentially. Instead:

1. You open the dictionary near the **middle** and check if the word you're looking for is before or after the current page.
2. Based on this, you either search the first half of the dictionary or the second half.
3. This process continues, halving the search space until you find the word.



Binary Search

- **Idea**

1. Start with a sorted array.

2. Find the middle element:

- If the middle element equals the target, return its position.

- If the middle element is greater than the target, search in the left half.

- If the middle element is less than the target, search in the right half

3. Repeat step 2 until the target is found or the search range becomes empty.

Binary Search: Iterative Version

- Assume sets of integers are sorted.
- Let us compare search_num to the element in the **middle** (the $(n/2)$ -th) of S)

```
def binary_search(sorted_set, search_num):  
    left = 0  
    right = len(sorted_set) - 1  
  
    while left <= right:  
        middle = (left + right) // 2 # Find the middle index  
  
        if sorted_set[middle] < search_num:  
            left = middle + 1 # Narrow search to the right half  
        elif sorted_set[middle] > search_num:  
            right = middle - 1 # Narrow search to the left half  
        else:  
            return middle # Return the index if found  
  
    return -1 # Return -1 if not found
```

```
searchnum = 18    size = 9
```

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

left=0

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=4

left=5

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=6

left=5 right=5

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=5

```
searchnum = 28    size = 9
```

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

left=0

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=4

left=5

right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=6

left=7 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=7

left=8 right=8

3	7	9	12	13	18	20	23	27
---	---	---	----	----	----	----	----	----

mid=8

Comparison between Recursion and Iteration

	Recursion	Iteration
Repetition	 By calling itself	 By using iterative structure (for, while)
Termination	 Base case	 Termination condition

In most cases, iteration and recursion can be **converted into each other.**

Binary Search: Recursive Version

Alg.: Binary Search(arr, searchnum, left, right)

1	if left = right	$O(1)$
2	if arr[left]= searchnum	$O(1)$
3	return left	$O(1)$
4	else	
5	return -1	$O(1)$
6	middle $\leftarrow$ (left + right)/2	$O(1)$
7		
8		
9		
10		
11		
12		

Question: What is the recursive formula?

Binary Search: Recursive Version

Alg.: Binary Search(arr, searchnum, left, right)

1	if left = right	$O(1)$
2	if arr[left]= searchnum	$O(1)$
3	return left	$O(1)$
4	else	
5	return -1	$O(1)$
6	middle $\leftarrow$ (left + right)/2	$O(1)$
7	if arr[middle] = searchnum	$O(1)$
8	return middle	$O(1)$
9	elseif arr[middle] < searchnum	$O(1)$
10	return BinarySearch(arr, searchnum, middle+1, right)	$O(?)$
11	else	
12	return BinarySearch(arr, searchnum, left, middle -1)	$O(?)$

Binary Search: Recursive Version

- Given the input size n , let $g(n)$ be the total number of basic operations executed in Binary Search

Alg.: Binary Search(arr, searchnum, left, right)

1	if left = right	}	We can still count the number of basic operations for this part	
2	if arr[left] = searchnum		}	The total number of basic operations executed is a constant independent of the input size n , we can use a to denote this
3	return left			
4	else	}	The total number of basic operations executed is a constant independent of the input size n , we can use a to denote this	
5	return -1			
6	middle $\leftarrow$ (left + right)/2	}	The total number of basic operations executed is a constant independent of the input size n , we can use a to denote this	
7	if arr[middle] = searchnum			
8	return middle	}	The total number of basic operations executed is a constant independent of the input size n , we can use a to denote this	
9	elseif arr[middle] < searchnum			
10	return BinarySearch(arr, searchnum, middle+1, right)	}	The total number of basic operations executed is a constant independent of the input size n , we can use a to denote this	
11	else			
12	return BinarySearch(arr, searchnum, left, middle-1)	}	The total number of basic operations executed is a constant independent of the input size n , we can use a to denote this	

We **either** run line 10 or 12, but **not** both. What is the number of basic operations that are executed by Line 10 or 12?

Analysis for Recursive Binary Search (i)

- Suppose that n is a power of 2.
- For $n = 1$, clearly
 - $g(n) = b$ for some constant b .
- $g(n)$ can be also defined recursively:
 - After executing a basic operations, we reduce the input size by half;
 - Then, we run the recursive binary search with input size at most $n/2$;
 - The running time on this sub-problem is at most $g(\frac{n}{2})$ by definition.



$$g(n) \leq a + g\left(\frac{n}{2}\right)$$

Analysis for Recursive Binary Search (ii)

- Given $g(n) = a + g\left(\frac{n}{2}\right)$, $g(1) = b$
 - What is $g(4)$ by using a and b to represent?

$$\begin{aligned}g(4) &\leq g(2) + a \\ &\leq g(1) + a + a \\ &\leq 2a + b\end{aligned}$$

- What is $g(n)$ by using a and b to represent if $n = 2^x$?

$$\begin{aligned}g(n) &\leq g\left(\frac{n}{2}\right) + a \\ &\leq g\left(\frac{n}{2^2}\right) + a + a \\ &\leq g\left(\frac{n}{2^3}\right) + a + a + a \\ &\leq g\left(\frac{n}{2^4}\right) + a + \underbrace{a + a + a + a}_{\log_2 n \text{ of them}} \\ &\leq g(1) + \underbrace{a + a + \cdots + a + a}_{\log_2 n \text{ of them}} = a \cdot \log_2 n + b = O(\log_2 n)\end{aligned}$$

Analysis for Recursive Binary Search (iii)

How to deal with general n that is not a power of 2?

- We can use a **rounding** approach. Let n' be the least power of 2 that is larger than n . It thus holds that $n' < 2n$.
- We then have:
- $f(n) \leq f(n')$
- $\leq a \cdot \log_2 n' + b$ (proved earlier)
- $< a \cdot \log_2(2n) + b$
- $= a \cdot (1 + \log_2 n) + b$
- $= a \cdot \log_2 n + a + b = O(\log n)$.

Exercise 5 — Linear Recursion

```
def mystery(n):  
    if n <= 1:  
        return 1  
    return mystery(n-1) + n
```

- What is the time complexity?

Exercise 6 — Two Recursive Calls

```
def mystery(n):  
    if n <= 1:  
        return 1  
    return mystery(n-1) + mystery(n-1)
```

- What is the time complexity?

Exercise 7 — Divide Into Two Halves

```
def mystery(n):  
    if n <= 1:  
        return 1  
    mystery(n//2)  
    mystery(n//2)
```

- What is the time complexity?

Exercise 8 — Three Recursive Calls

```
def mystery(n):  
    if n <= 1:  
        return 1  
    mystery(n-1)  
    mystery(n-1)  
    mystery(n-1)
```

- What is the time complexity?

Example: Fibonacci Number

- Fibonacci number is a sequence of number:
 - The n -th number in the sequence depends on the previous two numbers in the same sequence.
 - e.g., 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

$$f(x) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ f(n-1) + f(n-2), & n > 1 \end{cases}$$

n-th Fibonacci number: f(n)

```
if n is 0: return 0;
if n is 1: return 1;
if n is > 1:
    return f(n-1) + f(n-2);
```

Level	Nodes
-------	-------

0

 2^0
$$f(5)$$

1

 2^1 $f(4)$ $f(3)$

2

 2^2
$$f(3)$$
 $f(2)$ $f(2)$
$$f(1)$$
 $f(2)$ $f(1)$ $f(1)$ $f(0)$ $f(1)$ $f(C)$  $f(1)$ $f(0)$

Problem: Highly Repetitive Computation

n-th Fibonacci number: $f(n)$

```
if n is 0: return 0;
if n is 1: return 1;
if n is > 1:
    return f(n-1) + f(n-2);
```

Level Nodes

0

2^0

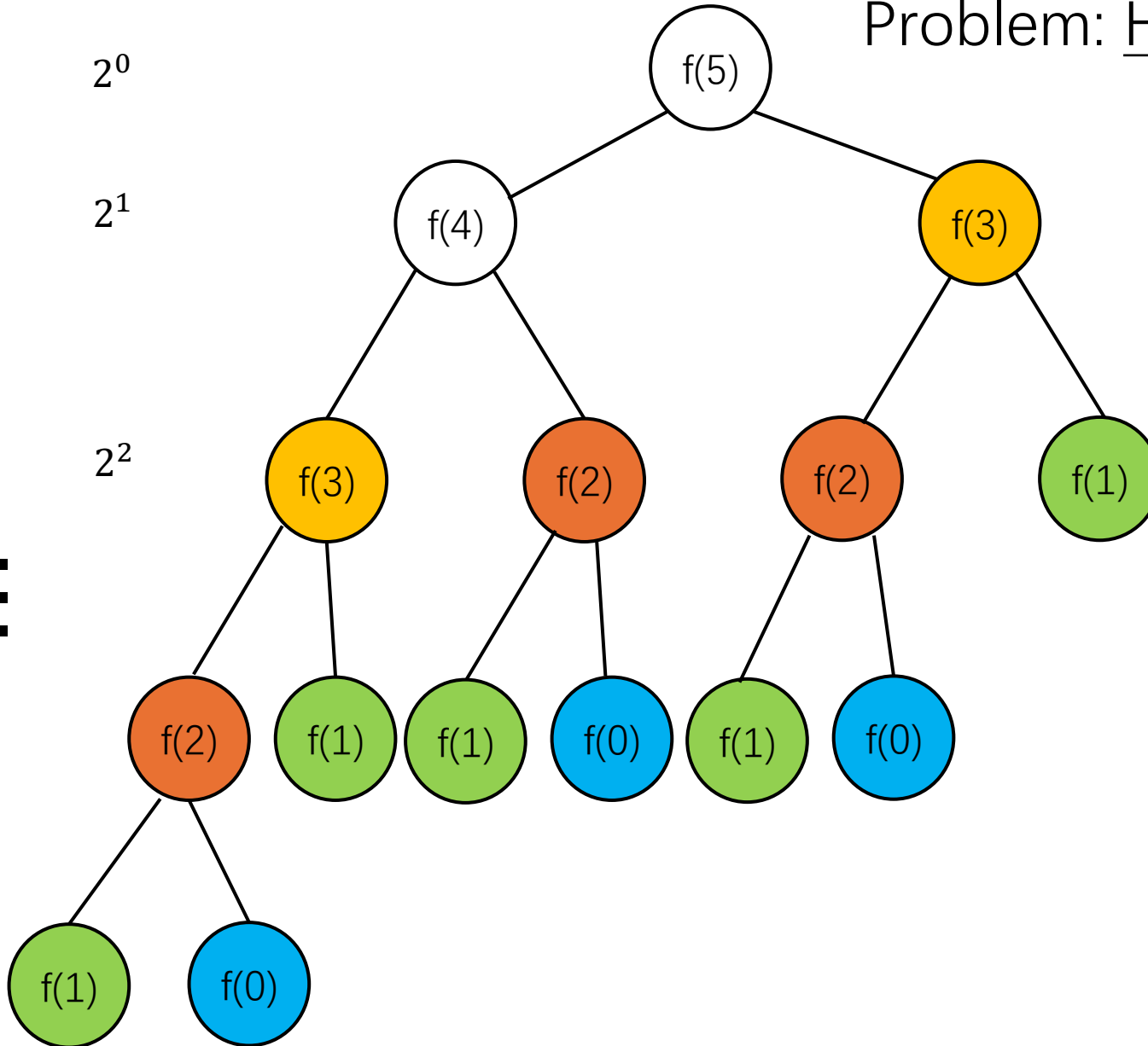
1

2^1

2

2^2

■
■
■



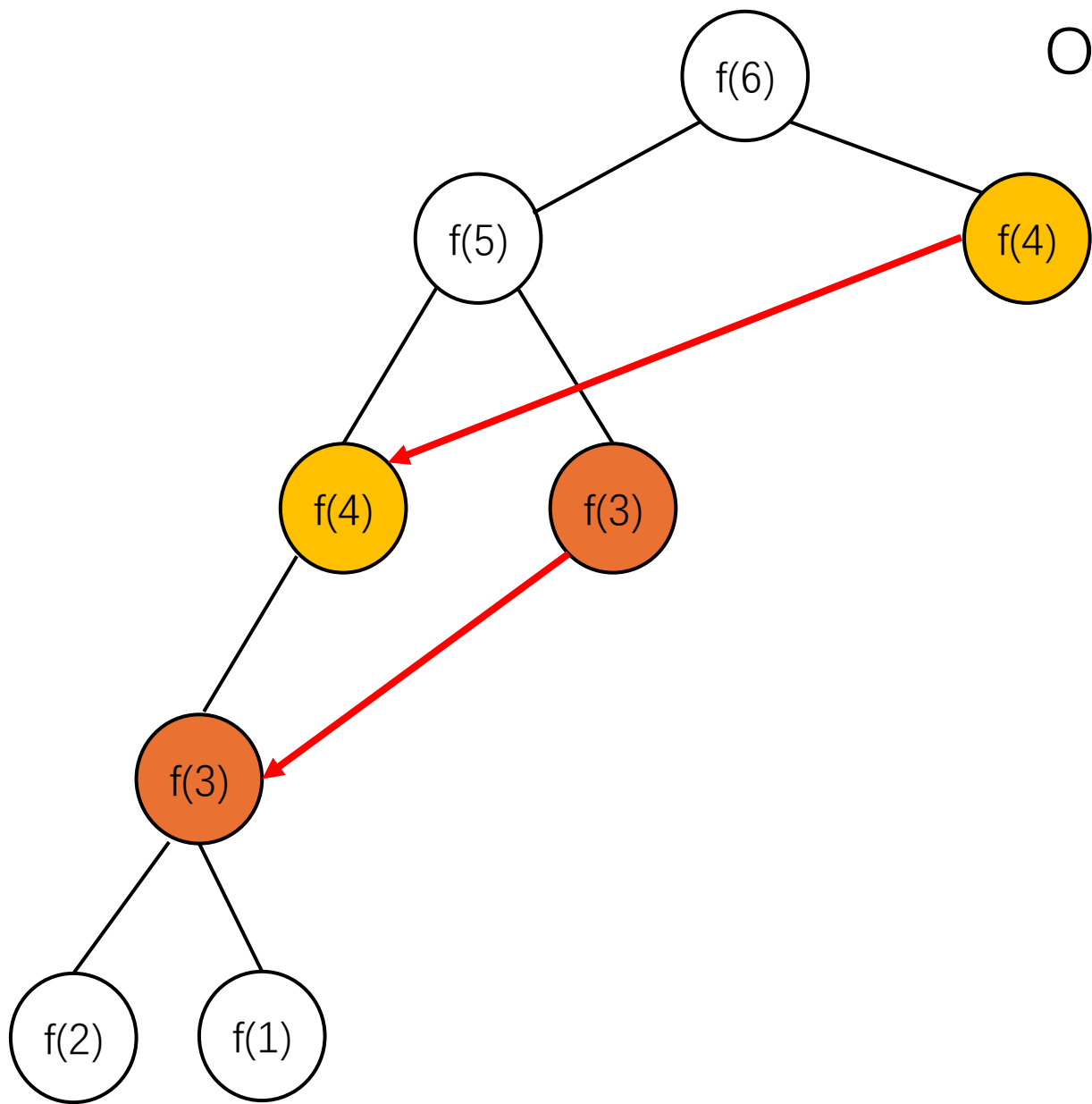
Problem: Highly Repetitive Computation

- Each node corresponds to one function call.
- For the first roughly $n/2$ levels, the number of nodes doubles.
- By level $n/2$, there are already a exponential number of nodes.
- So the running time is exponential.

n-th Fibonacci number: $f(n)$

```
if n is 0:  return 0;
if n is 1:  return 1;
if n is > 1:
    return f(n-1) + f(n-2);
```

Optimization Method: Memorization



- Use a list to store the intermediate result $f(i)$.
- Reduce the repetitive computation by computing $f(i)$ only once.
- Each level has at most 2 nodes; the total number of nodes is $O(n)$.
- The time complexity is $O(n)$.

n-th Fibonacci number: $f(n, \text{memo} = [])$

```
if n is 0: return 0;
if n is 1: return 1;
if n not in memo:
    memo[n] = f(n-1, memo) + f(n-2, memo)
return memo[n]
```

Exercise 9 — Design Your Own Recursive Function

- Design a recursive function:

`sum_digits(n)`

- that returns the **sum of digits** of an integer.
- Example:

`sum_digits(5382) = 5 + 3 + 8 + 2 = 18`

Requirements:

1. Define the **base case**
2. Define the **recursive formula**